

Linear Regression is a **supervised machine learning algorithm** used for **predicting continuous numerical values** (regression task). It tries to draw the best straight line through your data points so that this line can help us predict new values.

It assumes that there is an **approximately linear relationship** between the input features (independent variables) and the target variable (dependent variable).

Mathematically it tries to find the best values for **slope (m)** and **intercept (c)** in the equation:

$y = m \times x + c$ **Height and weight relationship or for every 100sq m 4 lakhs rupees increases so this is derived from linear regression**

Code used for predicting package on the basis of cgpa

```
plt.scatter(df['cgpa'],df['package'])
```

```
plt.xlabel('CGPA')
```

```
plt.ylabel('Package(in lpa)')
```

```
X = df.iloc[:,0:1]
```

```
y = df.iloc[:,-1]
```

```
from sklearn.model_selection import train_test_split
```

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=2)
```

```
from sklearn.linear_model import LinearRegression
```

```
lr = LinearRegression()
```

```
lr.fit(X_train,y_train) // training the model using inbuilt function
```

```
lr.predict(X_test.iloc[0].values.reshape(1,1))
```

Explain from scratch how the error function / cost function is calculated in Linear Regression.

Answer:

In Linear Regression we use **Mean Squared Error (MSE)** as the error function (also called cost function or loss function). It measures how far our predictions are from the actual values, on average.

The formula is:

$MSE = (1/n) \times \sum (y_i - \hat{y}_i)^2$ (or sometimes people write $J(\theta) = (1/(2n)) \times \sum (y_i - \hat{y}_i)^2$ — the 1/2 is just for mathematical convenience during derivative, both are correct)

Step-by-step calculation:

1. For each data point i:
 - Make prediction $\rightarrow \hat{y}_i = m \times x_i + b$
 - Calculate residual / error $\rightarrow e_i = \hat{y}_i - y_i$
2. Square each error $\rightarrow e_i^2$ (Why square?)
 - Makes all errors positive
 - Punishes large errors more $\rightarrow$ good because we hate big mistakes

- Makes the function smooth $\rightarrow$ easy for math/gradient descent)
- 3. Sum all squared errors $\rightarrow \sum e_i^2$
- 4. Divide by number of points $n \rightarrow$ get average squared error = MSE

Sometimes people also talk about **RMSE** = $\sqrt{\text{MSE}}$ $\rightarrow$ It is in the same unit as y (easier to interpret: “on average we are off by 5.2 marks”)

Why MSE and not just sum of absolute errors(mod kyun nhi leete)? $\rightarrow$ MSE is differentiable everywhere $\rightarrow$ very important for gradient descent $\rightarrow$ Squares make the optimization landscape smooth (bowl-shaped)

Our Own implementation of linear regression through self created formula and developed program

Python class having same implementation of scikiklearn as the inbuilt library of sklearn.linear model we imported linear regression

class MeraLR:

```
def __init__(self):
    self.m = None
    self.b = None

def fit(self,X_train,y_train)
    num = 0
    den = 0
    for i in range(X_train.shape[0]):
        num = num + ((X_train[i] - X_train.mean()*(y_train[i] - y_train.mean()))
        den = den + ((X_train[i] - X_train.mean()*(X_train[i] - X_train.mean()))

    self.m = num/den
    self.b = y_train.mean() - (self.m * X_train.mean())
    print(self.m)
    print(self.b)

def predict(self,X_test):
    print(X_test)
    return self.m * X_test + self.b
```

Multiple Linear Regression (MLR) is a **supervised machine learning algorithm** (and also a statistical method) used for **predicting a continuous numerical value** when we have **two or more independent variables** (features / predictors).

It extends simple linear regression by allowing multiple inputs to explain the output.

The general equation is:

$$y = w_1x_1 + w_2x_2 + w_3x_3 + \dots + w_n x_n + b$$

(or in vector form: $y = Xw + b$)

PYTHON CODE FOR MULTIPLE LINEAR REGRESSION

```
from sklearn.model_selection import train_test_split
```

```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size=0.2,random_state=3)
```

```
from sklearn.linear_model import LinearRegression
```

```
lr = LinearRegression()
```

```
lr.fit(X_train,y_train)
```

```
y_pred = lr.predict(X_test) // just predicting the model on the X_test which we separated using train test split.
```

```
lr.coef_
```

```
array([59.64518074, 13.20409431])// we are getting the value of weights B1 B2 that how much does the associated quantity has weights for the output y (kitta affect krega)
```

```
lr.intercept_
```

```
-7.55491251398082// this is the offset
```

Question: Why do we prefer Gradient Descent over the Normal Equation for large datasets in Linear Regression?

Answer:

We have two main ways to find the optimal parameters (weights θ or m & b) in Linear Regression that minimize the Mean Squared Error:

1. **Normal Equation** (closed-form solution): $\theta = (X^T X)^{-1} X^T y \rightarrow$ This gives the **exact mathematical best solution** in one step.
2. **Gradient Descent** (iterative): $\rightarrow$ Start with random $\theta \rightarrow$ repeatedly update θ using gradients until cost is low.

For **small to medium datasets** (say $< 10,000$ – $50,000$ rows and < 100 – $1,000$ features), Normal Equation is great because:

- It's exact (no approximation)
- No need to tune learning rate or iterations
- Very fast

But for **large datasets** (millions of rows, thousands/millions of features — very common in real ML today), we strongly prefer **Gradient Descent** (especially Stochastic GD or Mini-batch GD). The main reasons are:

- **Computational complexity / speed** Normal Equation requires computing $X^T X$ (which is $O(n \times m^2)$ where n = rows, m = features) + inverting an $m \times m$ matrix ($O(m^3)$). $\rightarrow$ When m (features) is large (e.g., $10,000+$), $O(m^3)$ becomes extremely slow and memory-intensive — sometimes impossible on normal machines.

Gradient Descent (batch version) per iteration is $O(n \times m)$, and even with many iterations (say $1,000$ – $10,000$), total cost is usually much lower than $O(m^3)$ when n is huge and m is moderate/large. $\rightarrow$ Stochastic GD is $O(1 \times m)$ per update $\rightarrow$ super fast even on billions of examples.

Polynomial Regression is a type of **regression model** that extends Linear Regression to capture **non-linear relationships** between the independent variable(s) and the dependent variable.

While Linear Regression assumes a straight-line relationship ($y = w_0 + w_1x$), Polynomial Regression models the relationship as an **n -th degree polynomial**:

$$y = w_0 + w_1x + w_2x^2 + w_3x^3 + \dots + w_n x^n$$

(where n is called the **degree** of the polynomial)

Even though the relationship with x is non-linear, the model is still **linear in the parameters** ($w_0, w_1, \dots, w_n$) — that's why we can use the same Normal Equation or Gradient Descent to fit it. In practice, we create new features ($x, x^2, x^3, \dots$) and then apply Multiple Linear Regression on them.

Why do we use Polynomial Regression?

1. It captures **curved/non-linear patterns** in data where simple linear regression fails (underfits).
2. It is still **simple, fast, and interpretable** compared to complex models like decision trees, random forests, or neural networks.
3. It is very useful for **small-to-medium datasets** where we suspect a smooth polynomial-like curve (growth curves, acceleration, diminishing returns, etc.).

It still uses the same math (least squares or gradient descent) — we just give it extra “fake” features like x^2, x^3 . That's why people say: “Polynomial Regression is still linear in the parameters — just non-linear in the input x .” we are concerned in calculating the relation of weights that is $\beta_0, \beta_1, \beta_2$ to get how much independent affects we don't look at x cause we already have it.